Background Information

Analyze the `Online Retail.csv` dataset and build a forecasting model to predict `'Quantity'` of products sold.

- Split the data into two sets based on the splitting date, `"2011-09-25"`. All data up to and including this date should be in the training set, while data after this date should be in the test set. Return a pandas DataFrame, `pd_daily_train_data`, containing, at least, the columns `"Country"`, `"StockCode"`, `"InvoiceDate"`, `"Quantity"`.
- Using your test set, calculate the Mean Absolute Error (MAE) for your forecast model for the `'Quantity'` sold? Return a double (float) named `mae`.
- How many units are expected to be sold during the week `39` of 2011? Store as an integer variable called `quantity_sold_w39`.

It's simple to buy any product with a click and have it delivered to your door. Online shopping has been rapidly evolving over the last few years, making our lives easier. But behind the scenes, e-commerce companies face a complex challenge that needs to be addressed.

Uncertainty plays a big role in how the supply chains plan and organize their operations to ensure that the products are delivered on time. These uncertainties can lead to challenges such as stockouts, delayed deliveries, and increased operational costs.

You work for the Sales & Operations Planning (S&OP) team at a multinational e-commerce company. They need your help to assist in planning for the upcoming end-of-the-year sales. They want to use your insights to plan for promotional opportunities and manage their inventory. This effort is to ensure they have the right products in stock when needed and ensure their customers are satisfied with the prompt delivery to their doorstep.

You are provided with a sales dataset to use. A summary and preview are provided below.

`Online Retail.csv`

Column	Description
`'InvoiceNo'`	A 6-digit number uniquely assigned to each transaction
`'StockCode'`	A 5-digit number uniquely assigned to each distinct product
`'Description'`	The product name
`'Quantity'`	The quantity of each product (item) per transaction

Column	Description
`'UnitPrice'`	Product price per unit
`'CustomerID'`	A 5-digit number uniquely assigned to each customer
`'Country'`	The name of the country where each customer resides
`'InvoiceDate'`	The day and time when each transaction was generated `"MM/DD/YYYY"`
`'Year'`	The year when each transaction was generated
`'Month'`	The month when each transaction was generated
`'Week'`	The week when each transaction was generated (`1` - `52`)
`'Day'`	The day of the month when each transaction was generated (`1` - `31`)
`'DayOfWeek'`	The day of the weeke when each transaction was generated (`0` = Monday, `6` = Sunday)

In [120…

```
# Import required libraries
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml import Pipeline
from pyspark.ml.regression import RandomForestRegressor
from pyspark.sql.functions import col, dayofmonth, month, year,  to_date, to
from pyspark.ml.feature import StringIndexer
from pyspark.ml.evaluation import RegressionEvaluator

# Initialize Spark session
my_spark = SparkSession.builder.appName("SalesForecast").getOrCreate()
```

```
25/03/18 19:10:44 WARN Utils: Service 'SparkUI' could not bind on port 4040.
Attempting port 4041.
```

In [121…

```
# Importing sales data
sales_data = my_spark.read.csv(
    "../data_raw/Online Retail.csv", header=True, inferSchema=True, sep=",")
sales_data.show(15)
```

```
[Stage 1:=====>                                                  (1 + 9) /
10]
```

```
+---------+---------+--------------------+--------+---------+----------+----
---------+-------------------+----+-----+----+---+---------+
|InvoiceNo|StockCode|         Description|Quantity|UnitPrice|CustomerID|
Country|        InvoiceDate|Year|Month|Week|Day|DayOfWeek|
+---------+---------+--------------------+--------+---------+----------+----
---------+-------------------+----+-----+----+---+---------+
|   536365|   85123A|WHITE HANGING HEA...|       6|     2.55|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536365|    71053| WHITE METAL LANTERN|       6|     3.39|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536365|   84406B|CREAM CUPID HEART...|       8|     2.75|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536365|   84029G|KNITTED UNION FLA...|       6|     3.39|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536365|   84029E|RED WOOLLY HOTTIE...|       6|     3.39|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536365|    22752|SET 7 BABUSHKA NE...|       2|     7.65|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536365|    21730|GLASS STAR FROSTE...|       6|     4.25|     17850|Unit
ed Kingdom|2010-01-12 08:26:00|2010|    1|   2| 12|        1|
|   536366|    22633|HAND WARMER UNION...|       6|     1.85|     17850|Unit
ed Kingdom|2010-01-12 08:28:00|2010|    1|   2| 12|        1|
|   536366|    22632|HAND WARMER RED P...|       6|     1.85|     17850|Unit
ed Kingdom|2010-01-12 08:28:00|2010|    1|   2| 12|        1|
|   536367|    84879|ASSORTED COLOUR B...|      32|     1.69|     13047|Unit
ed Kingdom|2010-01-12 08:34:00|2010|    1|   2| 12|        1|
|   536367|    22745|POPPY'S PLAYHOUSE...|       6|      2.1|     13047|Unit
ed Kingdom|2010-01-12 08:34:00|2010|    1|   2| 12|        1|
|   536367|    22748|POPPY'S PLAYHOUSE...|       6|      2.1|     13047|Unit
ed Kingdom|2010-01-12 08:34:00|2010|    1|   2| 12|        1|
|   536367|    22749|FELTCRAFT PRINCES...|       8|     3.75|     13047|Unit
ed Kingdom|2010-01-12 08:34:00|2010|    1|   2| 12|        1|
|   536367|    22310|IVORY KNITTED MUG...|       6|     1.65|     13047|Unit
ed Kingdom|2010-01-12 08:34:00|2010|    1|   2| 12|        1|
|   536367|    84969|BOX OF 6 ASSORTED...|       6|     4.25|     13047|Unit
ed Kingdom|2010-01-12 08:34:00|2010|    1|   2| 12|        1|
+---------+---------+--------------------+--------+---------+----------+----
---------+-------------------+----+-----+----+---+---------+
only showing top 15 rows
```

In [122…
```
# Convert InvoiceDate to datetime
sales_data = sales_data.withColumn("InvoiceDate", to_date(
    to_timestamp(col("InvoiceDate"), "d/M/yyyy H:mm")))
```

Aggregated daily sales:

- Grouped by Country, StockCode, and InvoiceDate.
- Summed up Quantity (total items sold per product per day).

In [123…
```
# Insert the code necessary to solve the assigned problems. Use as many code

# Aggregate data into daily intervals
daily_sales_data = sales_data.groupBy("Country", "StockCode", "InvoiceDate",
```

```
# Rename the target column
daily_sales_data = daily_sales_data.withColumnRenamed(
    "sum(Quantity)", "Quantity")
```

Split into Training & Testing Sets:

- Training data: All transactions before 2011-09-25.
- Test data: Transactions after 2011-09-25.

In [124…
```
# Split the data into two sets based on the spliting date, "2011-09-25".
# All data up to and including this date should be in the training set, whil

split_date_train_test = "2011-09-25"

# Creating the train and test datasets
train_data = daily_sales_data.filter(
    col("InvoiceDate") <= split_date_train_test)
test_data = daily_sales_data.filter(col("InvoiceDate") > split_date_train_te
```

In [125…
```
# Return a pandas Dataframe, pd_daily_train_data, containing, at least, the
pd_daily_train_data = train_data.toPandas()
pd_daily_train_data
```

Out[125…

	Country	StockCode	InvoiceDate	Year	Month	Day	Week	DayOfWeek	av
0	United Kingdom	22912	2010-01-12	2010	1	12	2	1	
1	France	22659	2010-01-12	2010	1	12	2	1	
2	United Kingdom	21544	2010-01-12	2010	1	12	2	1	
3	United Kingdom	21098	2010-01-12	2010	1	12	2	1	
4	Norway	85150	2010-01-12	2010	1	12	2	1	
...	...	...	...	...	...	...	...	...	
175447	United Kingdom	22646	2011-09-12	2011	9	12	37	0	
175448	United Kingdom	21218	2011-08-12	2011	8	12	32	4	
175449	United Kingdom	84006	2011-08-12	2011	8	12	32	4	
175450	United Kingdom	23141	2011-09-12	2011	9	12	37	0	
175451	Germany	21914	2011-09-12	2011	9	12	37	0	

175452 rows × 10 columns

Feature Engineering

- Extracted time-based features from InvoiceDate:
- Year, Month, Week, Day, DayOfWeek (helps the model capture seasonality).

In [126…

```
train_data = train_data.withColumn("Year", year("InvoiceDate"))
train_data = train_data.withColumn("Month", month("InvoiceDate"))
train_data = train_data.withColumn("Week", weekofyear("InvoiceDate"))
train_data = train_data.withColumn("Day", dayofmonth("InvoiceDate"))
train_data = train_data.withColumn("DayOfWeek", dayofweek("InvoiceDate"))
```

Model Training (Random Forest)

- Converted categorical features to numeric indexes (for Country and StockCode).
- Assembled feature columns into a vector.
- Trained a Random Forest model to predict product demand.

In [127…

```
from pyspark.ml.feature import VectorAssembler, StringIndexer
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml import Pipeline

# Creating indexer for categorical columns
country_indexer = StringIndexer(
    inputCol="Country", outputCol="CountryIndex").setHandleInvalid("keep")
stock_code_indexer = StringIndexer(
    inputCol="StockCode", outputCol="StockCodeIndex").setHandleInvalid("keep

# Selectiong features columns
feature_cols = ["CountryIndex", "StockCodeIndex", "Month", "Year",
                "DayOfWeek", "Day", "Week"]

# Using vector assembler to combine features
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

# Initializing a Random Forest model
rf = RandomForestRegressor(
    featuresCol="features",
    labelCol="Quantity",
    maxBins=4000
)

# Create a pipeline for staging the processes
pipeline = Pipeline(stages=[country_indexer, stock_code_indexer, assembler,

# Training the model
model = pipeline.fit(train_data)
```

```
25/03/18 19:10:59 WARN DAGScheduler: Broadcasting large task binary with siz
e 2007.9 KiB
25/03/18 19:11:00 WARN DAGScheduler: Broadcasting large task binary with siz
e 2.8 MiB
```

Model Evaluation

In [128…
```
# Predicted sales on test data
test_predictions = model.transform(test_data)
test_predictions = test_predictions.withColumn(
    "prediction", col("prediction").cast("double"))
```

- Computed Mean Absolute Error (MAE):
  - Lower MAE = Better model performance.

In [129…
```
from pyspark.ml.evaluation import RegressionEvaluator

### Provide the Mean Absolute Error (MAE) for your forecast? Return a double
# Initializing the evaluator
mae_evaluator = RegressionEvaluator(
    labelCol="Quantity", predictionCol="prediction", metricName="mae")

# Obtaining MAE
mae = mae_evaluator.evaluate(test_predictions)
mae
```

Out[129…
```
9.401993221091962
```

MAE = 9.4 tells you that, on average, your model's sales predictions deviate by ~9.4 units per product from actual sales.

Forecasting Future Sales

- Predicted sales volume for Week 39 of 2011.

In [130…
```
### How many units will be sold during the  week 39 of 2011? Return an integ

# Getting the weekly sales of all countries
weekly_test_predictions = test_predictions.groupBy("Year", "Week").agg({"pre
weekly_test_predictions.show()
```

```
[Stage 44:======================>                                  (4 + 6) /
10]
```

```
+----+----+------------------+
|Year|Week|   sum(prediction)|
+----+----+------------------+
|2011|  42| 95021.05921405388|
|2011|  44| 56753.56366904848|
|2011|  46|111205.41496660719|
|2011|  48|  68382.8888007228|
|2011|  45| 94352.10997823789|
|2011|  43| 92780.42358503533|
|2011|  39| 85221.64798936552|
|2011|  49| 64832.14471593006|
|2011|  47|102574.31257306121|
|2011|  41| 85677.31900424673|
|2011|  40| 49817.12812276992|
+----+----+------------------+
```

In [131…
```
# Finding the quantity sold on the 39 week.
promotion_week = weekly_test_predictions.filter(col('Week')==39)
promotion_week.show()
```

```
[Stage 50:=================>                                      (3 + 7) /
10]
+----+----+------------------+
|Year|Week|  sum(prediction)|
+----+----+------------------+
|2011|  39|85221.64798936552|
+----+----+------------------+
```

In [132…
```
# Storing prediction as quantity_sold_w30
quantity_sold_w39 = int(promotion_week.select("sum(prediction)").collect()[0
print(f"Predicted sales in Week 39: {quantity_sold_w39}")
```

```
[Stage 56:=======================================>                (7 + 3) /
10]
Predicted sales in Week 39: 85221
```

In [133…
```
actual_sales_w39 = test_data.filter(col('Week') == 39).agg({"Quantity": "sum
print(f"Actual sales in Week 39: {actual_sales_w39}")
```

```
[Stage 62:=====>                                                  (1 + 9) /
10]
Actual sales in Week 39: 93820
```

$$\text{Error Percentage} = \left(\frac{|93820 - 85221|}{93820} \right) \times 100 = 9.16\%$$

The model is off by ~9.16% in predicting total demand for Week 39. This is not bad for a first-pass forecasting model but could be improved.

In [134…

```
# Stop the Spark session
my_spark.stop()
```

Goal: Develop a machine learning model to predict product demand using historical sales data.

This project focused on:

1. Splitting the data into training and testing sets.
2. Feature engineering (extracting date-based patterns).
3. Training a Random Forest model for forecasting.
4. Evaluating the model using Mean Absolute Error (MAE).
5. Predicting demand for a future week.

✅ Processed time-series data for demand forecasting.
✅ Engineered time-based features (Year, Month, Week, Day).
✅ Trained a Random Forest model to predict sales.
✅ Evaluated model performance using MAE.
✅ Forecasted demand for a specific week (Week 39, 2011).